

- The assignment is due at Gradescope on 5/10/24.
- A LaTeX template will be provided for each homework. You are strongly encouraged to type your homework into this template using $\text{\LaTeX}$. If you are writing by hand, please fill in the solutions in this template, inserting additional sheets as necessary. This will help facilitate the grading.
- You are permitted to discuss the problems with up to 2 other students in the class (per problem); however, *you must write up your own solutions, in your own words*. Do not submit anything you cannot explain. If you do collaborate with any of the other students on any problem, please list all your collaborators in the appropriate spaces.
- Similarly, please list any other source you have used for each problem, including other textbooks or websites.
- *Show your work*. Answers without justification will be given little credit.
- Your homework is *resubmittable*. Please refer to the course syllabus on Canvas for a more detailed description of this. For any problem that you have not changed from your last submission, please make sure to indicate this in your submission to help our graders grade faster.

PROBLEM 1 (KNAPSACK AND LINEAR PROGRAMS) Recall the 0-1 Knapsack problem discussed in class: we are given a set of items with values and weights: $\{(v_i, w_i)\}_{i=1}^n$ and a weight budget W , and we are interested in finding the choice of items (i.e. the subset S of $[n]$) maximizing the sum of the values of the items:

$$\sum_{i \in S} v_i,$$

subject to the weight constraints:

$$\sum_{i \in S} w_i < W.$$

Consider the following linear program:

$$\begin{aligned} \max_{x \in \mathbb{R}^n} \quad & \sum_{i \in [n]} x_i v_i \\ & \sum_{i \in [n]} x_i w_i \leq W \\ & 0 \leq x_i \leq 1 \quad \forall i \in [n] \end{aligned} \tag{1}$$

Here, each variable x_i is meant to indicate whether or not we are including a specific item in our solution.

- Explain why the above linear program (1) doesn't solve the original 0-1 Knapsack problem, and provide an example instance of the 0-1 Knapsack problem that this LP gives an invalid solution for.
- Describe the problem, in words, that is solved by an instance of (1).
- Describe (in words) and give the pseudocode of an algorithm that finds the optimal assignment of x in (1).
- Compute the dual of (1).

for collaborators, Alex Fan helped me with part c of this question

Collaborators:

Solution:

solution

- The issue with the LP is that it sets X_i between 0 and 1, when it should be either 1 or 0. So the LP could potentially choose fractional items and optimize incorrectly, instead of selecting one item over another.
example: the weight constraint is 4 kg and value weights pairs are (6, 6), (2, 2), The Knapsack DP would return 2, because we have to fit 2 bags of weights 2 to reach 4. However, the LP will select a fraction of the item with a value of 6 such that its weight remains under 4.
- An instance of (1) represents the optimal amount of items to be taken in the knapsack, allowing for fractional quantities - only taking a part of an item.
- To find the optimal assignment of x in (1), I would use a greedy algo that chooses items by their value per weight. I would place items in the knapsack in decreasing order of value/weight, until the knapsack is filled in. For the last item placed in the knapsack, ie, the item that meets/exceeds W , I would only take a fraction of the item so the added weight meets the constraint.

Algorithm 1 Pseudo code for Knapsack - fractional items

Input: Set $S: (v_i, w_i)_{i=1}^n$, weight constraint: W

Output: The optimal assignment of x in (1)

Let A be an array of the items in S

Sort(A) by decreasing value/weight

Let counter = 0

for item in A **do**

if $w_i < W$ **then:** (Add item to knapsack)

 Counter+ = 1

$W - = w_i$

else(add fraction of item to knapsack)

 counter+ = W/w_i

Return knapsack and counter

end if

end for

- (d) I used the matrix form we were taught in class. If we are given $\max C^T x$, we can model constraints as $Ax \leq b, x_i \geq 0$. The dual of this form is represented as: $\min y^T b$ and $y^T A \geq C^T, y_i \geq 0$. From this, I found the dual to be $\min W y_1 + \sum_{i=2}^{n+1} y_i$, with the constraints: $w_i * y_1 + y_i \geq v_i, y_i \geq 0$

PROBLEM 2 (BOTTLENECK AND LOWER-BINDING EDGES) Let G be a flow network with integer edge capacities. An edge in G is upper-binding if increasing its capacity by 1 also increases the value of the maximum flow in G . Similarly, an edge is lower-binding if decreasing its capacity by 1 also decreases the value of the maximum flow in G .

- (a) Does every flow network G have at least one upper-binding edge? If so, sketch a proof. If not, give a counterexample.
- (b) Does every flow network G have at least one lower-binding edge? If so, sketch a proof. If not, give a counterexample.
- (c) Recall that an edge is saturated under a flow f if $f(e) = c(e)$. Prove or disprove: Given a flow network G , if an edge e is saturated under a max flow f , then e is an upper-binding edge.
- (d) Given a flow f on a flow network, we define the divergence $\text{div}(v)$ at a vertex as:

$$\sum_{(v,u) \in E} f_{vu} - \sum_{(w,v) \in E} f_{wv}.$$

Sketch a proof that given any subset $S \subseteq V$:

$$\sum_{v \in S} \text{div}(v) = \sum_{(u,v) \in E(S, \bar{S})} f_{uv} - \sum_{(u,v) \in E(\bar{S}, S)} f_{uv},$$

i.e. the total amount of flow out of S minus the total amount of flow into of S is equal to the total divergence inside of S .

(This is a discrete version of what is known as the divergence theorem in multivariable calculus, though understanding this is not needed to solve this problem).

- (e) Formally prove the following statement: Given a flow network G , if an edge e is an upper-binding edge, then e is saturated under **any** max flow f .

Collaborators:

Solution: Your solution here!

PROBLEM 3 (A FLOWY METRIC) Consider an undirected graph $G = (V, E)$ with capacities $c_e \geq 0$ on all edges $e \in E$. G has property that any cut in G has capacity at least 1. For example, a graph with a capacity of 1 on all edges is connected if and only if all cuts have capacity at least 1. However, in our case, c_e can be any non-negative number.

- (a) Show that for any two vertices $s, t \in V$, the max flow from s to t is at least 1.
- (b) Define the **length** of a flow f to be $\text{length}(f) = \sum_{e \in E} |f_e|$. Define the **flow distance** $d_{\text{flow}}(s, t)$ to be the minimum length of any s - t flow f that sends one unit of flow from s to t and satisfies all capacity constraints (i.e. $|f_e| \leq c_e$ for all $e \in E$). Write an LP that computes $d_{\text{flow}}(s, t)$.
- (c) Prove that if $c_e = 1$ for all $e \in E$, then $d_{\text{flow}}(s, t)$ is the length of the shortest path in G from s to t .
(Hint: let $d(s, t)$ be the length of the shortest path from s to t . You want to show $d_{\text{flow}}(s, t) = d(s, t)$ which is equivalent to showing that $d_{\text{flow}}(s, t) \leq d(s, t)$ and $d_{\text{flow}}(s, t) \geq d(s, t)$.)

Collaborators:

Solution: Your solution here!

PROBLEM 4 (ROAD TRIP 4: ROAD TRIP GAME) Lorenzo is going on another trip, and he's taking Ruimin along with him. They've already figured out the shortest path to take, and the optimal gas stations to stop at along the way - all they need left is some entertainment. Thankfully, Lorenzo has thought of a great game to keep them entertained.

The game is specified by an undirected, unweighted graph $G = (V, E)$ and two vertices $s, t \in V$. Lorenzo picks an edge $e \in E$, and Ruimin simultaneously picks a cut $S \subseteq V$ which must separate s and t i.e. $s \in S$ but $t \notin S$. We define the edge boundary of a cut S to be the set of edges crossing the cut:

$$E(S, \bar{S}) := \{uv \in E \mid u \in S, v \in \bar{S}\}.$$

Lorenzo wins the game if the edge he picked is in Ruimin's cut's edge boundary, and Ruimin wins otherwise. Formally, for the selected e and S , Lorenzo's payoff $P(e, S)$ is given by

$$P(e, S) = \begin{cases} 1 & \text{if } e \in E(S, \bar{S}), \\ 0 & \text{otherwise.} \end{cases}$$

Lorenzo is trying to maximize $P(e, S)$, and Ruimin is trying to minimize it.

We are interested in **mixed strategies** for this game: Lorenzo will assign each edge e some probability $p_e \geq 0$, and randomly select an edge accordingly. Similarly, Ruimin's strategy will involve randomly selecting a cut c , where each cut c is selected with probability p_c .

- (a) Lorenzo knows that he can't outsmart Ruimin, so he knows that whatever strategy he picks, Ruimin is just going to read his mind and figure it out. To determine his best strategy, he is going to solve the following LP:

$$\begin{aligned} \max_{\alpha, q} \quad & \alpha \\ \text{s.t.} \quad & \sum_{e \in C} q_e \geq \alpha \quad \forall C \in \mathcal{C} \\ & \sum_e q_e = 1 \\ & 0 \leq q_e \quad \forall e \in E(G) \end{aligned}$$

where $\mathcal{C}$ is the set of s - t cuts in G . Explain, in words, how this LP represents the game.

- (b) Ruimin knows that Lorenzo is a cheater, so he knows that whatever strategy he picks, Lorenzo is going to sneakily take a glance at it before deciding on his own strategy. In that case, write the linear program that Ruimin must solve to determine his optimal strategy. (Assume that Lorenzo picks his strategy after Ruimin.)
- (c) Demonstrate that these programs are each other's dual. (You may use the fact that the dual of a dual program is the primal.)
- (d) Part (c) implies the existence of some probability p such that:
- If Lorenzo plays optimally, his probability of winning is at least p , even if Ruimin knows his strategy.
 - If Ruimin plays optimally, Lorenzo's probability of winning is at most p , even if Lorenzo knows Ruimin's strategy.

Provide a short argument (2 sentences) that part (c) implies this.

- (e) Sketch a proof that the p from part (d) is $p = \frac{1}{L}$ where L is the length of a shortest path from s to t . This is Lorenzo's expected payoff from the game if both players play optimally, and is known as the **value** of the game.

Collaborators:

Solution: Your solution here!